

RP11: Mixed-Integer Hyperparameter Optimization using CMA-ES

Jon Rodriguez Gonzalez and Eneko Basauri Miñana

University of the Basque Country (EHU)
`{jrodriguez247, ebasauri002}@ikasle.ehu.eus`

Abstract. This report investigates mixed-integer hyperparameter optimization using the Covariance Matrix Adaptation Evolution Strategy (CMA-ES) and its discrete-variable adaptation, CMAwM. The objective function was defined as $1 - \text{accuracy}$ of a neural network trained with mixed continuous and discrete hyperparameters. The experiments aimed to evaluate whether CMAwM’s ability to handle discrete search spaces provides an advantage over the standard CMA-ES. Contrary to expectations, the results show that CMA-ES achieved better average performance than CMAwM.

1 Problem definition

The family of black-box optimization problems is characterized by the fact that the underlying structure of the problem is unknown. The only accessible information consists of the input data x and their corresponding output values y . Consequently, many optimization methods that rely on gradients or analytical forms are inapplicable. In the literature, a variety of optimization techniques can be found to solve these types of problems. Among them, we highlight those based on Covariance Matrix Adaptation, such as the state-of-the-art Covariance Matrix Adaptation Evolution Strategy (CMA-ES) algorithm[1].

CMA-ES is a stochastic, population-based optimization algorithm designed for continuous, non-linear, and non-convex problems, and particularly well suited for ill-conditioned, non-separable or rugged problems. The algorithm iteratively samples candidate solutions from a multivariate normal distribution and adapts its mean, step size, and covariance matrix based on past evaluations, enabling efficient exploration and exploitation without requiring gradient information. Moreover, CMA-ES is particularly well suited for optimizing expensive black-box objective functions[2], such as those arising in hyperparameter optimization of Neural Networks (NNs).

Hyperparameter optimization is a crucial step in the design and training of NNs, as the choice of hyperparameters—such as learning rate, batch size, number of layers, among others—can significantly impact model performance. However, evaluating each hyperparameter configuration typically requires training the NN, which is computationally expensive and time-consuming. Consequently, efficient

black-box optimization methods like CMA-ES are particularly valuable, as they can navigate the high-dimensional, non-convex hyperparameter space effectively while minimizing the number of costly evaluations.

Although CMA-ES is primarily designed for continuous optimization, many real-world problems—including hyperparameter optimization—contain variables that are discrete, categorical, or integer-valued. In such cases, a simple approach is to discretize continuous samples by rounding them to the nearest valid value; however, this naive strategy is generally suboptimal.

To better handle such mixed-integer problems, several extensions of CMA-ES have been proposed. For instance, CMA with Weighted Mutations (CMAwM)[3] [4] [5] modifies the sampling and adaptation mechanisms to account for discrete, integer, or categorical variables, providing a more effective and flexible approach than simple rounding. These variants allow CMA-based algorithms to retain their exploration and adaptation capabilities while addressing the challenges posed by non-continuous search spaces.

In this report, our goal is to empirically evaluate the performance of standard CMA-ES with naive discretization and its mixed-variable extension CMAwM, in the context of mixed-integer hyperparameter optimization. To this end, we optimize a set of hyperparameters of a convolutional NN trained on the MNIST dataset. By comparing the two algorithms we aim to determine whether CMAwM provides superior performance over the standard CMA-ES approach.

2 Steps for Solving the Problem

As mentioned in the previous section, to address the mixed-integer hyperparameter optimization task, we implement and compare two CMA-based optimization algorithms: the standard CMA-ES with naive discretization and the mixed-variable extension CMAwM. Both methods are applied to the optimization of a convolutional NN trained on the MNIST dataset. The implementation relies on the *cmes*[6] Python library for the optimization algorithms and *PyTorch* library for the NN definition, training, and evaluation.

Selection of Hyperparameters

To formulate the optimization problem, we define a set of five hyperparameters that significantly influence the training dynamics and generalization performance of the CNN. These hyperparameters include both continuous and discrete variables, making this problem a suitable test case for mixed-integer optimization algorithms.

– Number of layers — *Discrete*

Determines the depth of the convolutional NN. Increasing the number of

layers allows the model to learn more complex hierarchical representations, at the cost of higher computational requirements.

Bounds: [1, 4]

– **Kernel size** — *Discrete*

Defines the receptive field of each convolutional filter. Larger kernels capture broader spatial patterns, whereas smaller kernels focus on fine-grained local details.

Bounds: [3, 7]

– **Batch size** — *Discrete*

Specifies the number of training samples processed in a single forward backward pass. Smaller batch sizes introduce more gradient noise but may improve generalization; larger batch sizes offer more stable gradient estimates but increase memory usage.

Bounds: [32, 128]

– **Learning rate** — *Continuous*

Represents the step size of the optimizer during gradient descent. A larger learning rate accelerates convergence but risks overshooting minima, while a smaller rate ensures stability but may slow down learning.

Bounds: [10^{-5} , 10^{-1}]

– **Weight decay** — *Continuous*

A regularization parameter added to the optimizer to discourage too large weights, thereby reducing overfitting and improving generalization.

Bounds: [0.0, 0.1]

Define the Objective Function

The optimization problem is formulated as a minimization task, where the objective function

$$f(x) = 1 - \text{accuracy}(NN(x))$$

returns the validation error obtained after training a CNN with hyperparameters specified by the vector x on the MNIST dataset.

Additionally, in the case of naive discretization CMA-ES, before each evaluation, the sampled continuous vector x must be mapped to valid model parameters. Continuous variables are used as is, while integer variables are obtained by rounding the corresponding continuous values to their nearest valid integer within predefined bounds.

Overview of the *cmaes* Optimization Framework

Both optimization algorithms follow the standard iterative CMA framework, composed of three primary phases:

1. **Ask:** Sample a population of candidate solutions $\mathbf{x}_1, \dots, \mathbf{x}_\lambda$ from the current multivariate normal distribution.

2. **Evaluate:** Compute the objective function $f(\mathbf{x}_i)$ for each candidate.
3. **Tell:** Update the distribution parameters $(\mu, \sigma, \mathbf{C})$ based on the evaluated fitness values.

The key distinction between the two optimization strategies lies in how they handle the discrete components of x . In naive discretization CMA-ES, continuous samples are generated as usual but are rounded to the nearest valid integer before evaluation. The CMA-ES update step, however, still operates in the continuous domain, using the unrounded samples to adapt the covariance matrix. In contrast, CMAwM introduces the parameter *steps*, which enables the algorithm to internally account for discrete variables. This mechanism ensures that neighboring discrete options continue to be explored, preventing the search distribution from collapsing prematurely around a single integer value.

3 Experiments proposed

The objective of the experiments is to empirically validate the hypothesis that, under mixed-integer hyperparameter optimization the CMAwM algorithm yields better results than naive discretization CMA-ES.

To test this, both optimization variants will be executed under identical conditions, using the same maximum number of generations (10 generations). Additionally, two different initial values for σ (0.5 and 1.0) will be tested. Each configuration will be run for five independent iterations to obtain more statistically robust results. Although a larger number of trials per configuration would increase statistical confidence, the number of runs is limited by the high computational cost of the objective function and the available computational resources.

Additionally, as a secondary experiment, we aimed to increase the influence of the discrete variables to assess whether this change would further highlight the advantages of the CMAwM algorithm. Specifically, we modified the interpretation of the *batch size* variable by treating it as the total amount of training data used for the NN, rather than merely the batch size per iteration. This adjustment significantly amplified the impact of this discrete variable on the objective function, introducing additional noise and variability into the optimization process.

4 Results of the experiments

In this section, we present and analyze the results obtained from the experiments described in the previous section.

4.1 Main experiment

Figures 1a and 1b illustrate the evolution of the best validation error across generations for both algorithms under two different initial step-size settings ($\sigma = 0.5$

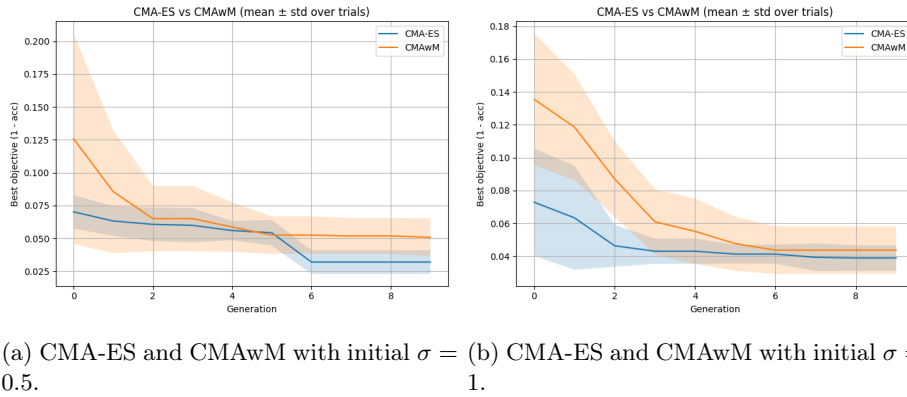


Fig. 1: Comparison of CMA-ES and CMAwM performance for different initial σ values. The plots show the mean and standard deviation of the best achieved 1-accuracy of the NN over 5 independent runs.

and $\sigma = 1.0$). Each curve represents the average performance across five independent runs, and the shaded regions denote the standard deviation, indicating the variability of the results.

Overall, the results show that CMA-ES achieved better performance than CMAwM in both experimental settings. CMA-ES achieving a mean of 0.032 in the case $\sigma = 0.5$ and 0.039 in the case $\sigma = 1$. In contrast, CMAwM achieved 0.051 in the case $\sigma = 0.5$ and 0.044 in the case $\sigma = 1$. It should be noted, however, that in both cases—as shown in the referenced figures—the initial population for CMAwM happened to be significantly worse than that of CMA-ES, which could have had an impact on its overall performance. As previously mentioned, additional iterations would be required to draw more definitive conclusions.

4.2 Additional experiment

In this secondary analysis, we aimed to investigate whether increasing the influence of discrete variables—specifically, by redefining the *batch size* parameter to represent the total amount of training data—would accentuate the differences between the two algorithms. Under this setting, the optimization landscape became significantly noisier and more sensitive to changes in that given discrete parameter.

The results, illustrated in Figures 2 and 3, indicate that in this modified scenario CMAwM achieved faster convergence. This improvement can be attributed to CMAwM’s ability to better manage discrete search spaces through its weighted mutation mechanism, which maintains exploration among neighboring integer values rather than collapsing prematurely around a single choice.

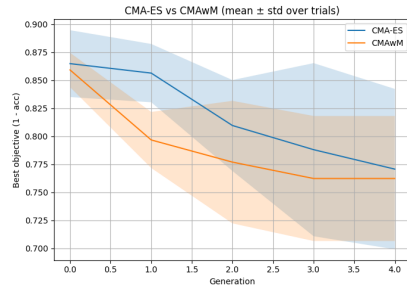


Fig. 2: Validation error using only one batch of size *batch size* over 5 trials with $\sigma = 0.5$.

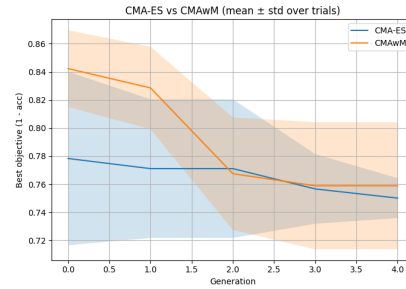


Fig. 3: Validation error using only one batch of size *batch size* over 5 trials with $\sigma = 0.5$.

5 Conclusions

This report investigated the performance of two optimization algorithms—CMA-ES with naive discretization and its mixed-variable extension CMAwM—in the context of mixed-integer hyperparameter optimization for a convolutional NN trained on the MNIST dataset.

The main experimental results revealed that, contrary to initial expectations, the standard CMA-ES algorithm consistently outperformed CMAwM in terms of the best achieved validation error under both tested step-size settings ($\sigma = 0.5$ and $\sigma = 1.0$). This outcome suggests that, for the chosen problem setup the additional complexity introduced by CMAwM did not yield a measurable advantage. One possible explanation is that the optimization landscape for the defined hyperparameter space was sufficiently smooth with regard to those discrete variables for the naive rounding strategy of CMA-ES to perform adequately, or that CMAwM’s adaptation mechanisms require a larger number of evaluations to fully realize their benefits.

However, the additional experiment—where the influence of discrete variables was deliberately increased—showed that CMAwM exhibited faster convergence in noisier search space. This finding supports the idea that CMAwM’s design is advantageous in scenarios where discrete or categorical parameters exert a stronger influence on the objective function, highlighting its robustness in mixed-variable settings.

Overall, the experiments demonstrate that while standard CMA-ES remains a strong baseline for mixed-integer optimization tasks, CMAwM offers potential benefits when the discrete components of the problem are more dominant.

Future work should involve running the algorithms for a larger number of generations and independent iterations to obtain more statistically reliable re-

sults. Moreover, further experiments could include testing on additional datasets, exploring different hyperparameter sets and NN architectures, and conducting statistical significance analyses to draw more conclusive insights into the relative performance of the optimization methods.

References

1. Hansen, N., Ostermeier, A.: Adapting arbitrary normal mutation distributions in evolution strategies: The covariance matrix adaptation. In: Proceedings of the 1996 IEEE International Conference on Evolutionary Computation, pp. 312–317 (1996)
2. Hansen, N., Auger, A., Ros, R., Finck, S., Pošík, P.: Comparing results of 31 algorithms from the black-box optimization benchmarking BBOB-2009. In: Proceedings of the 12th Annual Conference Companion on Genetic and Evolutionary Computation (GECCO 2010), pp. 1689–1696. ACM, New York (2010)
3. Auger, A., Hansen, N.: A restart CMA evolution strategy with increasing population size. In: Proceedings of the IEEE Congress on Evolutionary Computation (CEC 2005), vol. 2, pp. 1769–1776. IEEE, Edinburgh (2005)
4. Uchida, K., Hamano, R., Nomura, M., Saito, S., Shirakawa, S.: CMA-ES for discrete and mixed-variable optimization on sets of points. In: International Conference on Parallel Problem Solving from Nature, pp. 236–251. Springer, Cham (2024)
5. Hamano, R., Saito, S., Nomura, M., Shirakawa, S.: CMA-ES with margin: Lower-bounding marginal probability for mixed-integer black-box optimization. In: Proceedings of the Genetic and Evolutionary Computation Conference (GECCO 2022), pp. 639–647. ACM, New York (2022)
6. Nomura, M., Shibata, M.: CMAES: A simple yet practical Python library for CMA-ES. CoRR, abs/2402.01373 (2024)